A MECHANISM FOR PERSISTENTLY NAMING TOPOLOGICAL ENTITIES IN HISTORY-BASED PARAMETRIC SOLID MODELS

(Topological ID System)

Jiri Kripac

Autodesk, Inc.
111 McInnis Parkway, San Rafael, CA 94903

Email: jkripac@autodesk.com

ABSTRACT

One of fundamental problems in history-based parametric solid modeling systems is to identify topological entities of solid models in such a way that the *same* entities can still be identified after the models have been *reevaluated* from the sequential *history* of modeling operations.

The Topological ID System systematically assigns *IDs* (names) to topological entities (faces, edges, vertices) in solid models. When the solid model is edited and then automatically reevaluated from the history of modeling operations, the IDs of topological entities in the old version of the model are *mapped* to IDs of the *corresponding* topological entities in the new version of the model. This mapping defines the correspondence between topological entities in both models.

1. INTRODUCTION

Feature-based parametric modeling systems usually define models by a *sequential history of operations*. Whenever the user performs a modeling operation, for instance chamfers an edge, makes a slot, etc., the parametric system not only performs the operation on the current model but also records all the defining parameters of the operation into a history file. This sequential history of operations unambiguously defines the model and the parametric system can automatically recreate the model anytime by *reevaluating* this stored *history* of operations.

Feature-based modeling systems associate additional information with solid models and their topological entities such as faces, edges and vertices. These associations must be *persistent*, i.e. when the model is automatically reevaluated from its history of operations, the topological associations must be preserved and not lost. Typical examples of associations are:

- Feature dependencies within the model. For example, the information that a hole lies in a particular face, that a fillet or a chamfer is attached to a particular edge or a vertex, etc.
- Dimensions and annotations attached to faces, edges and vertices of the model (model-drawing associativity).
- Variational constraints associated with the model.

The fundamental problem is how to identify the topological entities of the solid model in such a way that these entities can still be identified even after the model has been reevaluated from the history of operations [Hof93].

Unfortunately, directly using pointers to topological entities is not good enough because pointers are transient. Whenever the parametric modeler reevaluates the model, it creates a brand new model and the original pointers become invalid.

Any simple scheme, such as sequentially enumerating all entities in the model, is also not enough because when the model is edited and automatically reevaluated, the topology of the model may change and the enumeration becomes invalid. Even if the topology of the model doesn't change and only the geometry is modified, the modeling algorithms such as Boolean operations may create or modify the topological entities of the model in a different order and consequently the enumeration cannot be preserved.

The *Topological ID System*, presented in this paper, tries to solve this problem. It systematically assigns IDs to topological entities (faces, edges, vertices) in a solid model. When the solid model is being reevaluated from its history of operations, the Topological ID System *maps* IDs of topological entities from the old version of the solid model to IDs of *corresponding* topological entities in the new version of the solid model[1]. This mapping defines the correspondence between topological entities in the old and new model.

[1] Instead of saying "old version of the solid model" or "new version of the solid model", we will simply say "old model" or "new model".

Permission to copy without fee all or part of this material is granted provided that the copies are not made or distributed for direct commercial advantage, the ACM copyright notice and the title of the publication and its date appear, and notice is given that copying is by permission of the Association of Computing Machinery. To copy otherwise, or to republish, requires a fee and/or specific permission.
Solid Modeling '95, Salt Lake City, Utah USA
© 1995 ACM 0-89791-672-7/95/0005...$3.50

If the topology of the model changes, this mapping may not be one-to-one, e.g. one edge in the old model may correspond to more than one edge in the new model or to no edge at all. The mapping algorithm should be able to handle these cases but should avoid finding false correspondences between unrelated topological entities in both models.

The parametric modeling system communicates with the Topological ID System through a narrow interface:

1) When the modeling system wants to make a topological entity persistent, i.e. identifiable after the model reevaluation, it provides a pointer to the entity to the Topological ID System and obtains a *PersId* (Section 6).

2) When the model is being reevaluated, the modeling system gives this *PersId* back to the Topological ID System and obtains pointers to topological entities in the new model. This second step is repeated for every reevaluation of the model and every time pointers to topological entities in the new model are obtained.

The Topological ID System consists of five major components:

- IDs to name faces, edges and vertices of solid models and steps in the history of modeling operations (Section 2).
- An additional data structure (*FaceIdGraph*) associated with each model that keeps the information about how faces of the model were created, split, merged and deleted (Section 3). This additional information is needed when mapping IDs from the old model to the new model.
- A mapping algorithm for mapping IDs from the old version of the model to the new version of the model (Section 4).
- An algorithm for reassigning *FaceIds* to faces in the new version of the model consistently with the assignment in the previous version of the model (Section 5) and for updating the *FaceIdGraph* when a modeling operation changes the topology of the model. For detailed information see [Kri94].
- A data structure associated with each solid model keeping a record for each modeling step. Each record contains *PersIds* of the topological entities which were recorded at this modeling step. When this modeling step is being reevaluated, the Topological ID System automatically remaps these *PersIds* to correspond to topological entities in the new model.

2. TYPES OF IDS

2.1. *StepId*

Each step in the history of operations is assigned a *StepId*. This *StepId* uniquely identifies the particular step (modeling operation) and remains unchanged when the model is reevaluated.

2.2. *FaceId*

FaceIds are the principal types of IDs. Our view is that regularized solid models are 3D volumes bounded by a set of faces. Edges and vertices are considered to be intersections of bounding faces and *EdgeIds* and *VertexIds* are therefore defined in terms of their adjacent *FaceIds*.

Each face in the boundary of a solid model is assigned a unique *FaceId*. A *FaceId* of face *f* is defined by three components:

$$FaceId(f) = [\ stepId, faceIndex, surfaceType\ ]^2$$

where:

stepId	*StepId* of the step during which face *f* was created.
faceIndex	Face index of the newly created face during the step *stepId*.
surfaceType	Type of the underlying surface of the face.

2.3. *EdgeId*

Edges are considered to be intersections of two or more faces; *EdgeId* of edge *e* is therefore defined in terms of *FaceIds* of faces surrounding the edge (Figure 1):

$$EdgeId(e) = [\ adjFaceIds, endFaceIds_{0,1}, edgeGeomType\ ]$$

where:

adjFaceIds	Ordered cyclic list of *FaceIds* of faces sharing edge *e*.
$endFaceIds_0$	Unordered set of *FaceIds* of faces at one end of edge *e*.
$endFaceIds_1$	Unordered set of *FaceIds* of faces at the other end of edge *e*.
edgeIntersCode	Edge intersection code.

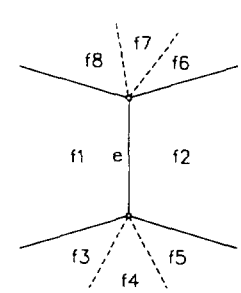

Figure 1: *EdgeId(e) = [*

adjFaceIds:	$<f_1, f_2>^3$,
$endFaceIds_0$:	$\{f_3, f_4, f_5\}^4$,
$endFaceIds_1$:	$\{f_6, f_7, f_8\}$,
edgeIntersCode:	*kAnyEdgeIntersCode]*

---

[2] *[elem, elem, ...]* indicates an ordered set of elements.
[3] *<elem, elem, ...>* indicates a cyclically ordered list of elements.

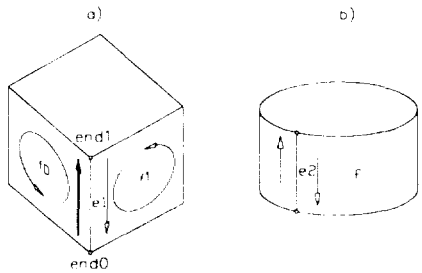

Figure 2:

a) The orientable edge e_1 is shared by two faces, $adjFaceIds(e_1) = <f_0, f_1>$. The end 0 and end 1 of the edge are defined with respect to face f_0.

b) The non-orientable edge e_2 is shared by a single face f which is used in both directions.

An edge which has at least one adjacent face included only once in the *adjFaceIds* set is said to be *orientable* because we can use this face to distinguish between the end 0 and the end 1 of the edge (Figure 2a). Edges which have all their adjacent faces included more than once in the *adjFaceIds* set are *non-orientable*, i.e. we cannot distinguish between end 0 and end 1 of the edge looking only at the *adjFaceIds* set of that edge (Figure 2b).

If an edge is orientable, $endFaceIds_0$ corresponds to the end 0 of the edge and $endFaceIds_1$ corresponds to the end 1.

The *adjFaceIds* and $endFaceIds_{0,1}$ sets of *FaceIds* provide local topological information about an edge. However, in some cases, this local topological information is not enough to uniquely identify the edge.

The *edgeIntersCode* field of the *EdgeId* provides the additional information to disambiguate edges that are otherwise topologically the same, i.e. they have exactly the same *adjFaceIds* and $endFaceIds_{0,1}$. A common example are the two edges coming from the subtraction of two cylinders (Figure 3). These edges have their adjacent faces exactly the same and there is no local topological information that could be used to distinguish between these two edges.

The *EdgeIntersCode* is based on the relative position of the *surfaces* of the adjacent faces of the edge, not on the absolute coordinates. This makes the *EdgeIntersCode* invariant under rigid motion and scaling transformations.

When the surfaces intersect at more than one intersection curve, each of these intersection curves is assigned a different *EdgeIntersCode*. The *EdgeId* then receives the *EdgeIntersCode* of the intersection curve the edge lies on. In the above example, the *EdgeIds* of the two edges are distinguished by their different *EdgeIntersCode* values.

[4] *{elem, elem, ...}* indicates a unordered set of elements.

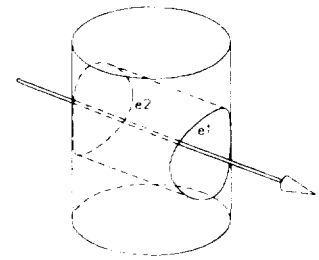

Figure 3: The *EdgeIntersCode* of edges *e1* and *e2* is based on the direction of the axis of the subtracted cylinder and distinguishes these two edges.

2.4. *VertexId*

Vertices are considered to be intersections of faces; *VertexId* of vertex *v* is therefore defined by a set of *FaceIds* of faces intersecting at this vertex:

$$VertexId(v) = [\ adjFaceIds,\ vertexIntersCodes\ ]$$

where:

adjFaceIds	Set of *FaceIds* of faces sharing vertex *v*.
vertexIntersCodes	Vertex intersection code for each face in *adjFaceIds* set.

In the case of manifold vertices, all faces sharing the vertex form a simple ordered loop around the vertex and *adjFaceIds* is an ordered loop of *FaceIds* of faces sharing this vertex. In the case of non-manifold *vertices*, *adjFaceIds* is an unordered set of *FaceIds* of all faces sharing this vertex.

The set of adjacent faces provides local topological information about a vertex. However, in some special cases, such as in the case of the two vertices on a degenerate "lemon" or "apple" torus, this information is not enough to uniquely define the vertex.

In order to disambiguate *VertexIds* of these special vertices, an additional *VertexIntersCode* attribute is provided for each face in the *adjFaceIds* set. This information specifies where the vertex lies with respect to the surface of the face.

3. EVOLUTION OF FACES

The Topological ID System associates with each solid model a data structure called a *FaceIdGraph* which is updated every time the topology of the model changes.

The *FaceIdGraph* is a directed acyclic graph. Its nodes, called *FaceIdNodes*, represent faces in the model, both faces that currently exist in the model and faces that once existed in the model but don't exist in the current model any more because they were split, merged or deleted as a result of a modeling operation applied to the model.

The incoming edges to a *FaceIdNode* represent information about the ancestors of this face and the outgoing edges represent what happened to the face (Figure 4):

- If there is more than one incoming edge to a *FaceIdNode*, it means that the face was the result of *merging* two or more faces. The faces being merged ceased to exist in the model and were replaced with the new face.
- If there is more than one outgoing edge from a *FaceIdNode*, it means that the face has been *split* into several new faces. The original face ceased to exist and was replaced with several new faces.
- If there is no incoming edge to a *FaceIdNode*, it means that the face has been *created directly*, for instance, when faces of a primitive solid are created. This is the most common case.
- If there is one outgoing edge from a *FaceIdNode* to a special node named *deletedFaceIdNode*, it means that the whole face has been *deleted*.
- If there is no outgoing edge from a *FaceIdNode*, it means that the face still exists in the current model. Such faces are called *living faces*. Any other *FaceIdNodes* in the graph represent faces that once existed in the model but were split, merged or deleted.

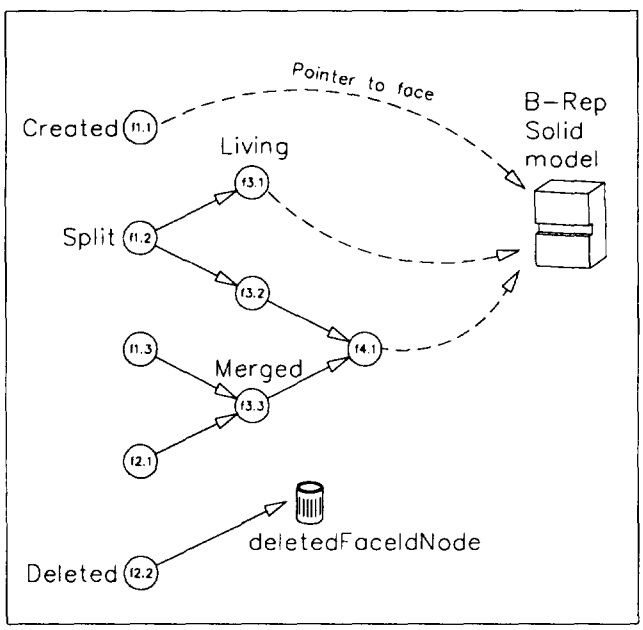

Figure 4: In this *FaceIdGraph* faces $f_{1.1}$[5], $f_{1.2}$, $f_{1.3}$, $f_{2.1}$ and $f_{2.2}$ have been created, then $f_{1.2}$ has been split into $f_{3.1}$ and $f_{3.2}$, $f_{1.3}$ and $f_{2.1}$ have been merged into $f_{3.3}$ and $f_{2.2}$ has been deleted. Finally, $f_{3.2}$ and $f_{3.3}$ have been merged into $f_{4.1}$. Only $f_{1.1}$, $f_{3.1}$ and $f_{4.1}$ exist in the current model.

[5] $f_{m.n}$ denotes a face created during *StepId m* and with index *n* w.r.t. to the step *m*.

4. MAPPING TOPOLOGICAL IDS

The mapping algorithm maps the given old ID (*FaceId, EdgeId, VertexId*) of a topological entity in the old version of the solid model to a set of new IDs of the *corresponding* topological entities in the new version of the model.

The main requirements are:

- *Intuitive behavior*, meaning that the algorithm must find the correspondences between old and new topological entities which users would expect.
- Handling *curved surface* topology, e.g. edges and vertices which are intersections of non-planar surfaces, handling special cases such as cone apices, etc.
- Handling *topology changes* between the old and the new model but avoiding finding false correspondences between unrelated entities in both models.

The mapping is implemented by a function *mapId()*:

mapId: [oldId,
oldFaceIdGraph,
newFaceIdGraph,
newModel,
IdMapRequest] → [newIdSet, IdMapStatus]

where:

oldId	ID of a topological entity in the old model which is going to be mapped to the new model.
oldFaceIdGraph	*FaceIdGraph* of the old model.
newFaceIdGraph	*FaceIdGraph* of the new model.
newModel	Solid B-Rep data structure of the new model.
IdMapRequest	Required quality of mapping (Section 4.4).
newIdSet	The resulting set of IDs of topological entities in the new model corresponding to the old topological entity *oldId*.
IdMapStatus	Achieved "quality" of mapping.

The achieved "quality" of mapping may be:

kExactIdMapStatus	The *oldId* has been mapped to exactly the same new ID. It means that the topology of the entity didn't change between the old and the new version of the model.
kUniqueIdMapStatus	The *oldId* has been mapped to exactly one new ID. It means that the entity changed between the old and new model but the mapping algorithm was able to track the change and find a single corresponding entity in the new model.

kSomeIdMapStatus The *oldId* has been mapped to more than one new ID. It means that the old entity has been split into several pieces in the new model.

kFailedIdMapStatus The *oldId* didn't map to any new ID. It means that no reasonably similar topological entity has been found in the new model.

4.1. *FaceId* Mapping

Algorithm: Map a given *FaceId* from the old model to the new model.

Input: *oldFaceId*
oldFaceIdGraph, newFaceIdGraph
IdMapRequest

Output: *newFaceIdSet*
IdMapStatus

Steps:

- **Exact-Equality-Test:**

Try to find a face with the *oldFaceId* in the new model. If it exists, return the *oldFaceId*. Otherwise, use the next two steps.

- **Backward-Search in the *oldFaceIdGraph*:**

Starting from the node with the given *oldFaceId*, go backward in the *oldFaceIdGraph*, until you reach a *FaceId* that is also present in the *newFaceIdGraph*[6]. Select all such *FaceIds*, i.e. process all branches in parallel.

- **Forward-Search in the *newFaceIdGraph*:**

For those *FaceIds* found during the Backward-Search, go forward in the *newFaceIdGraph* until you reach nodes that have no successors, i.e. they are living nodes corresponding to faces that currently exist in the new model. Select all such nodes.

Return *FaceIds* of these living nodes. □

Figures 5 and 6 show examples of behavior of the *FaceId* mapping algorithm.

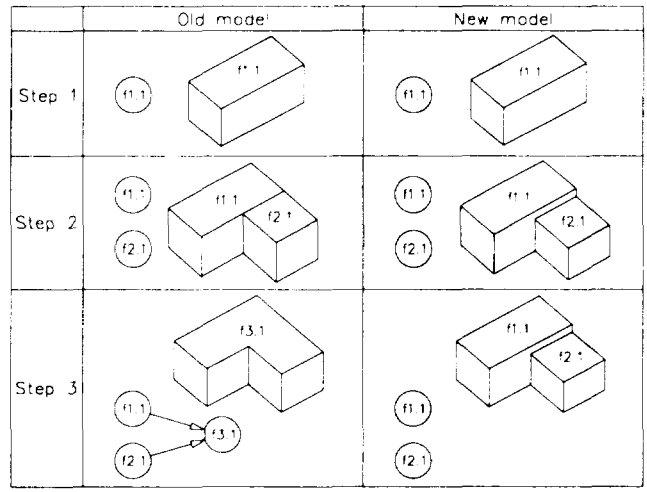

Figure 5: Two boxes have been united. In the old model the top faces $f_{1.1}$ and $f_{2.1}$ have been merged into face $f_{3.1}$. In the new model these faces stay as two different faces.

When face $f_{3.1}$ in the old model is mapped to the new model, the Backward-Search finds two common faces $f_{1.1}$ and $f_{2.1}$. The Forward-Search returns the same faces because both $f_{1.1}$ and $f_{2.1}$ are living faces in the new model.

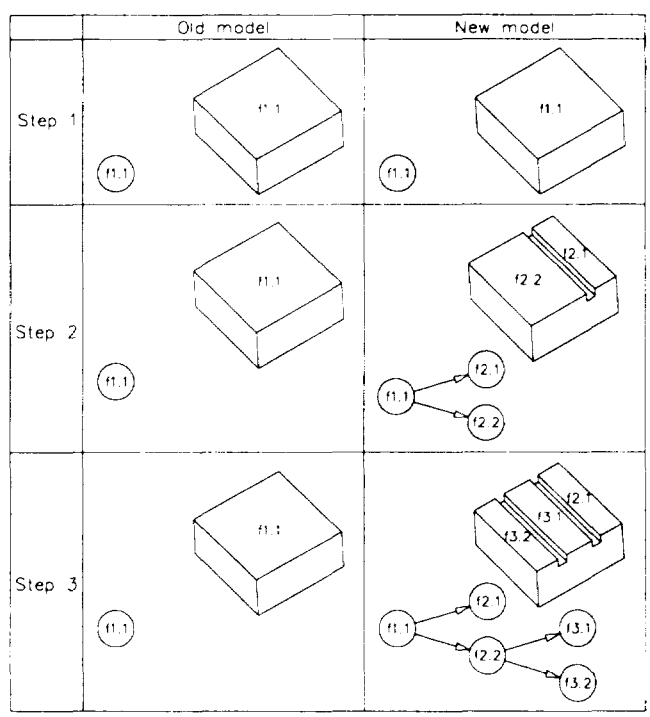

Figure 6: When face $f_{1.1}$ in the old model is mapped to the new model, the Backward-Search selects the same face $f_{1.1}$. The Forward-Search selects all living successors of this common face $f_{1.1}$, i.e. faces $f_{2.1}$, $f_{3.1}$ and $f_{3.2}$.

[6]The purpose of the Backward-Search is to find *FaceIds* in the *oldFaceIdGraph* which resulted in the given *oldFaceId* and which are also present in the *newFaceIdGraph*. These *common FaceIds* mean that there was a *common step* in the history of the old model and the history of the new model during which these common *FaceIds* were created.

4.2. *EdgeId* Mapping

Algorithm: Map a given *EdgeId* from the old model to the new model.

Input: *oldEdgeId*
oldFaceIdGraph, newFaceIdGraph
B-Rep of the new model
IdMapRequest

Output: *newEdgeIdSet*
IdMapStatus

Steps:

- **Exact-Equality-Test:**

Try to find an edge with the *oldEdgeId* in the new model. If it exists, return the *oldEdgeId*.

Otherwise use a sequence of tests to select the matching edges. First, a set of *candidateEdges* is created and then additional rejection tests try to reject "less matching" candidate edges and leave just a single best one.

- **AdjFaceIds-Test:**

Select all edges in the new model which have at least two adjacent *FaceIds* matching[7] adjacent *FaceIds* of the *oldEdgeId*. Store all the edges found in the *candidateEdges* set.

The goal is to obtain exactly one matching edge, if possible. If one or zero candidate edges have been found, return the *EdgeId* of this candidate edge or return an empty set, respectively. If more than one candidate edge has been found, continue with the rejection tests.

- **EndFaceIds-Test:**

Select the edges from the *candidateEdges* set which have at least one *FaceId* at each end of the candidate edge matching a *FaceId* at each end of the *oldEdgeId*. A special case is when one or both *endFaceIds* sets of one edge are empty, in which case the corresponding *endFaceIds* set of the other edge must also be empty.

If both edges are oriented, the algorithm takes into account the orientation of both *EdgeIds*. It distinguishes which end of each *EdgeId* is end 0 and which is end 1 and makes sure that the correct ends of both *EdgeIds* are compared.

The EndFaceIds-Test may eliminate some candidate edges from the *candidateEdges* set. If the reduced *candidateEdges* set contains exactly one edge, return *EdgeId* of this single edge. If no edge would be left, use the original *candidateEdges* set before the EndFaceIds-Test was performed. Continue with the next test.

- **EdgeIntersCode-Test:**

Select the edges from the *candidateEdges* set whose *EdgeIntersCode* is equal to the *EdgeIntersCode* of the *oldEdgeId*.

The EdgeIntersCode-Test may eliminate some edges from the *candidateEdges* set. If no edge would be left in the *candidateEdges* set, use the original *candidateEdges* set before the EdgeIntersCode-Test was performed.

Return *EdgeIds* of the edges left in the *candidateEdges* set.□

Figures 7, 8 and 9 show examples of the behavior of the *EdgeId* mapping algorithm.

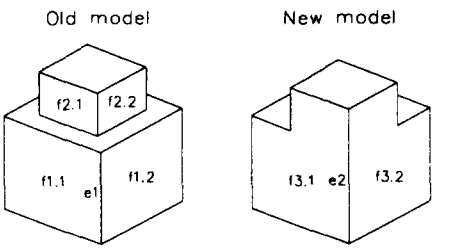

Figure 7: Edge e_1 of the old model has been selected. In the new model, the top box has been moved. The AdjFaceIds-Test finds new edge e_2 because $f_{1.1}$ maps to $f_{3.1}$ and $f_{1.2}$ maps to $f_{3.2}$.

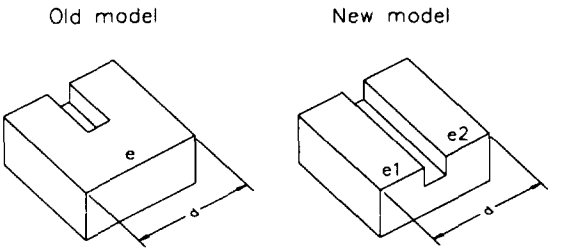

Figure 8: Edge e of the old model has been selected. In the new model, the slot has been extended so that the edge was split. The AdjFaceIds-Test selects two candidate edges e_1, e_2. Neither the EndFaceIds-Test nor the EdgeIntersCode-Test selects just a single edge, therefore both e_1 and e_2 are returned.

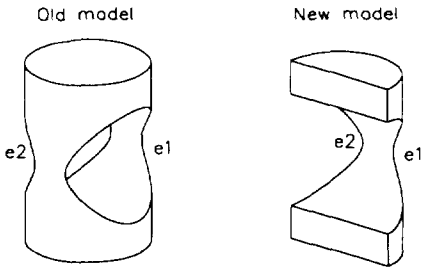

Figure 9: Edge e_1 of the old model has been selected. In the new model, the cylinder has been sectioned. The AdjFaceIds-Test selects e_1 and e_2 as candidate edges. Because there is more than one candidate edge, EndFaceIds-Test is tried. Neither candidate edge satisfies the EndFaceIds-Test, therefore the EdgeIntersCode-Test is used. This test accepts edge e_1 and rejects edge e_2.

[7] Saying that an old *FaceId* "matches" a new *FaceId* means that the old *FaceId* maps to a set of *FaceIds* which contains the new *FaceId*.

4.3. *VertexId* Mapping

Algorithm: Map a given *VertexId* from the old model to the new model.

Input: *oldVertexId*
oldFaceIdGraph, newFaceIdGraph
B-Rep of the new model
IdMapRequest

Output: *newVertexIdSet*
IdMapStatus

Steps:

- **Exact-Equality-Test:**

Try to find a vertex with the *oldVertexId* in the new model. If it exists, return the *oldVertexId*. Otherwise use the Best-Similarity-Rating-Test.

- **Best-Similarity-Rating-Test:**

Traverse all vertices in the new model and select the vertex or vertices whose *VertexIds* have the highest similarity rating with the *oldVertexId*. Return *VertexIds* of these selected vertices.

The similarity rating is based on the similarity between *adjFaceIds* and *vertexIntersCodes* of the *oldVertexId* and vertices in the new model. The similarity rating algorithm is analogous to the algorithm used for determining the similarity between two *FaceIdLoops* (see Section 5). □

Figure 10 shows an example of the behavior of the *VertexId* mapping algorithm.

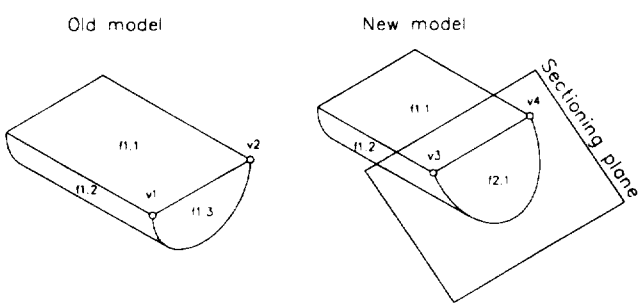

Figure 10: Vertex v_1 in the old model has been selected. In the new model, the object has been sectioned. Using the Best-Similarity-Rating-Test, the mapping algorithm selects vertex v_3 in the new model, whose similarity rating is *kSubsetMatch*.

4.4. Required Quality of Mapping

The *IdMapRequest* input argument specifies the required "quality" of mapping. The possible requests are:

kExactIdMapRequest The given old ID is required to map to *exactly the same* ID in the new model.

kUniqueIdMapRequest The given old ID is required to map to *exactly one* new ID.

kSomeIdMapRequest The given old ID is required to map to *at least one* new ID.

kAnyIdMapRequest *Any* result of mapping is accepted, including the case where the given old ID couldn't map to any new ID.[8]

The mapping algorithm maps the given old ID to new IDs. It then checks whether the achieved quality of the mapping (*IdMapStatus*) satisfies the required quality of the mapping (*IdMapRequest*). If yes, the resulting new IDs are returned[9]. If not, the system tries to meet the requested quality by performing the mapping to a *previous state* of the new model.

The new model is *rolled back* to its previous state and the mapping is performed again. If the quality of the mapping to the rolled-back model satisfies the *IdMapRequest*, copies of the topological entities found are made and returned along with the mapped IDs. Otherwise, the rollback continues until the *IdMapRequest* is satisfied or until no more rollback is possible. Finally, the new model is rolled forward to its original position before any rollback started.

Example:

The model (Figure 8, old model) was created in three steps:

Step 1: Create a base feature.
Step 2: Create a slot.
Step 3: Attach a dimension to edge *e*, record *PersId*[10] of edge *e* into the history file.

When the length of the slot is changed, the model is reevaluated. When *Step 3* is being reevaluated, the parametric system needs to find an edge or edges in the new version of the model corresponding to edge *e* in the old version of the model. Depending on the value of *IdMapRequest*, the result will be:

- If *kAnyIdMapRequest* is specified, the two edges e_1 and e_2 are returned.
- If *kUniqueIdMapRequest* is specified, the edge *e* before it was split is returned.

In our example, the parametric system would most likely specify *kUniqueIdMapRequest* to receive exactly one edge *e* and use its geometry to place the dimension. The whole edge *e* before it was split would be dimensioned, as shown in Figure 8, new model.

In other cases, e.g., if the modeling system wanted to fillet the edge, *kAnyIdMapRequest* would be specified and two edges e_1 and e_2 obtained. Both edges would then be filleted.

5. MAPPING SPLIT FACES

The split face mapping algorithm maps *FaceId* of a face which resulted from splitting a face in the old version of the

---

[8]This is the default value.
[9]*kAnyIdMapRequest* is always satisfied and the result of the mapping is instantly returned.
[10]Section 6.

model to a *corresponding* split face in the new version of the model. The correspondence between the old and the new split faces is determined based on the similarity between their surrounding faces. A *FaceIdLoop* class is introduced for this purpose (Figure 11).

The *FaceIdLoop* of face f is a cyclic list:

$$FaceIdLoop(f) = < elem(e_1), elem(e_2), ... >$$

$$elem(e) = [\, faceIdSet, edgeIntersCode \,]^{11}$$

where:

faceIdSet	Set of *FaceIds* of adjacent faces sharing the edge e with face f.
edgeIntersCode	*EdgeIntersCode* of edge e. This information is necessary to disambiguate some topologically similar cases where all the split faces have the same loops of adjacent *FaceIds* (Figure 12).

When a face is bounded by exactly one loop of edges, this loop is used for the *FaceIdLoop*. When the face is bounded by more than one loop of edges, the goal is to use the most dominant loop for the *FaceIdLoop*. The most dominant loop of planar faces is their outer loop. If the face is non-planar and is bounded by more than one loop of edges, all faces of all loops are taken and the *FaceIdLoop* is considered to be an unordered set.

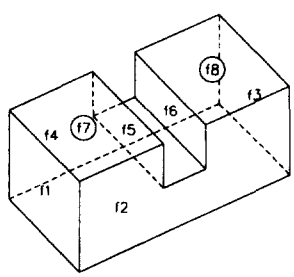

Figure 11: $FaceIdLoop(f_7) = <f_1, f_2, f_5, f_4>$
$FaceIdLoop(f_8) = <f_2, f_3, f_4, f_6>$

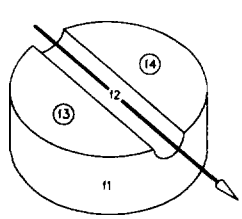

Figure 12: *FaceIdLoops* of both f_3 and f_4 consist of the same faces $<f_1, f_2>$. In order to disambiguate these two *FaceIdLoops*, *EdgeIntersCode* information is used. The *EdgeIntersCode* of the edge between faces f_3, f_2 is different from the *EdgeIntersCode* of the edge between faces f_4, f_2 because the direction of the subtracted cylinder is taken into account.

[11] In the examples we will simplify the notation and write *elem(e)* as just the adjacent face f_i.

Let us now define a measure of similarity between two *FaceIdLoops*. The similarity rating must return the highest value if the two *FaceIdLoops* are exactly the same and should return a lower value if the loops are not the same but are similar, which happens when the topology of the model changes.

Algorithm:	Measure similarity between two *FaceIdLoops*.
Input:	*oldFaceIdLoop* *newFaceIdLoop* *oldFaceIdGraph, newFaceIdGraph*
Output:	*kExactMatch* *kSubsetMatch* *k3ElemMatch* *k2ConsecElemMatch* *k2ElemMatch* *k1ElemMatch* *kDisjointMatch*

The algorithm returns one of the following similarity values, listed in the descending order from the highest similarity rating to the lowest:

kExactMatch: The loops match exactly. This means that each *elem* of one loop matches an *elem* of the other loop.

The cyclic ordering of *elems* in both loops is *significant*, i.e. the position of the matching *elems* in both loops as well as the direction of both loops is required to be the same.[12]

kSubsetMatch: One loop is a subset of another, meaning that all *elems* of one loop match *elems* in the other loop. The cyclic ordering of *elems* in both loops is significant.

k3ElemMatch: At least three *elems* in one loop match *elems* in the other loop. The cyclic ordering of *elems* in both loops is significant.

k2ConsecElemMatch: Two *elems* in one loop match two *elems* in the other loop. These two matching *elems* are positioned consecutively in both loops, so there is no gap between them. The order of the two matching *elems* in both loops *is* significant.

k2ElemMatch: Similar to the above case, but there is a gap between the two matching *elems* in either one or both loops. The order of the two matching *elems* in both loops is *not* significant[13].

k1ElemMatch: Just one *elem* in one loop matches an *elem* in the other loop.

[12] If both *FaceIdLoops* are oriented, the cyclic ordering of both loops is considered, otherwise the loops are treated as unordered sets.

[13] Even if not immediately evident, the *k2ElemMatch* is substantially weaker than *k2ConsecElemMatch* because the information about the orientation of both loops is lost.

kDisjointMatch: The two loops are completely disjoint. □

Example (Figure 13):

$FaceIdLoop(f_6) = <f_1, f_2, f_3, f_5>$
$FaceIdLoop(f_7) = <f_1, f_5, f_3, f_4>$

$FaceIdLoop(f_x) = <f_1, f_8, f_2, f_9, f_3, f_5>$
$FaceIdLoop(f_y) = <f_1, f_5, f_3, f_{10}, f_4, f_{11}>$

The similarity rating table between the old and the new split faces is:

	$FaceIdLoop(f_x)$	$FaceIdLoop(f_y)$
$FaceIdLoop(f_6)$	*kSubsetMatchRating*	*k2ElemMatchRating*
$FaceIdLoop(f_7)$	*k2ElemMatchRating*	*kSubsetMatchRating*

This means that $FaceIdLoop(f_6)$ is most similar to $FaceIdLoop(f_x)$ and $FaceIdLoop(f_7)$ is most similar to $FaceIdLoop(f_y)$. Therefore face f_6 maps to face f_x and face f_7 maps to face f_y.

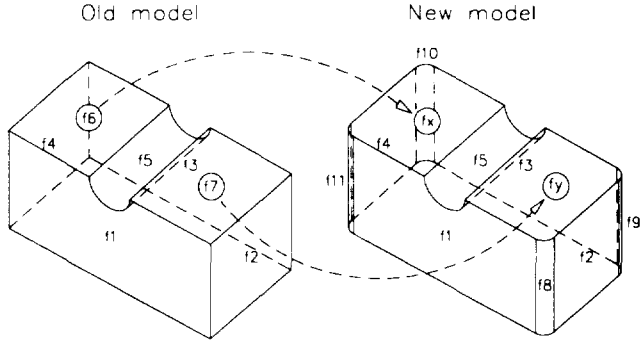

Figure 13: In the old model, the top face of the box has been split by a slot into two faces f_6 and f_7. In the new model, the vertical edges of the box have been filleted before the slot operation is reevaluated.

6. API

In order to record a topological entity (face, edge, vertex) as persistent, the application records the pointer to this topological entity and obtains a *PersId*[14]. The *PersId* does *not* change when the model is reevaluated. The application then uses this *PersId* to refer to the recorded topological entity in the future.

To record a topological entity *entPtr* and obtain its *PersId*:

```
PersId persId = TopId::record(entPtr,req15);
```

[14]The *PersId* is basically an index of a record which keeps ID (*FaceId*, *EdgeId*, *VertexId*) of the recorded entity and other data. When the *PersId* is remapped, the stored ID may be remapped to none or to more than one new ID.

[15]The *IdMapRequest req* says how the entity is to be remapped in the future (Section 4.4).

When the model is being reevaluated, the *PersId* is automatically remapped so that it refers to topological entities in the new version of the model. To obtain all entities in the new model to which a persistent topological entity with the given *PersId* has been remapped, the following method is used:

```
newEnts = TopId::get(persId);
```

The returned *newEnts* set is a set of pointers to all topological entities to which the given *PersId* refers. Because of remapping, one *PersId* may refer to any number of topological entities in the new model.

The Topological ID System associates a data structure with each solid model keeping one record for each modeling step. Each record keeps the *StepId* of the step and *PersIds* of all topological entities which were recorded at this modeling step. For this purpose any modeling step must be enclosed between:

```
TopId::stepBegin(stepId);
Perform the modeling operation;
TopId::stepEnd(oldModel16,newModel, ...);
```

This way the newly created faces are assigned *FaceIds*, the *FaceIdGraph* of the new model is updated and *PersIds* saved with this modeling step are remapped.

Here are typical steps the parametric history-based modeling system may use when it performs an operation for the first time and when it automatically reevaluates this operation:

Recording an Operation:

1. Obtain a pointer to a topological entity, e.g., ask the user to pick an edge to be filleted.
2. Record this topological entity and obtain its *PersId*.
3. Save the *PersId*.
4. Use the topological entity, e.g. fillet the edge.

Reevaluating the Operation from Stored Parameters:

1. Take the saved *PersId*.
2. From the *PersId* obtain pointers to topological entities in the new model.
3. Check whether exactly one new entity has been obtained. If not, it means that the remapping didn't map the old entity to exactly one new entity. React appropriately to this situation.
4. Use the topological entity, e.g. fillet the edge.

7. CONCLUSION

The presented paper tries to come up with a more robust approach to the problem of persistently identifying topological entities in history-based parametric solid models. We see the superiority of the presented solution in the following areas:

- Ability to disambiguate topologically similar entities.
- Ability to handle curved surface geometry and edges and vertices that are intersections of curved surfaces.

[16]The *oldModel* (actually only its *FaceIdGraph*) is needed for the ID mapping algorithms (Section 4).

- Ability to handle split faces and merged faces, particularly to distinguish between the individual faces resulting from splitting another face.
- Object-oriented software architecture with a narrow application interface.

Finally some possible directions for further work:

- The presented mechanism has been designed for manifold and non-manifold solid models. A natural extension would be to make the system handle general non-manifold structures.
- An interesting research topic might be to formalize the notions of "similarity" and "mapping" between topological models.

8. ACKNOWLEDGEMENT

I would like to thank to Ravi Krishnaswamy for reading the manuscript and making many helpful suggestions.

9. REFERENCES

[BP93] Buchanan, S A and Pennington A, 'Constraint Definition System: A Computer-Algebra Based Approach to Solving Geometric Constraint Problems' *Computer-Aided Design* Vol 25 No 12 (December 1993) pp 741-750

[DZ93] Duan, W, Zhou, J and Lai, K, 'FSMT: A Feature Solid-Modeling Tool for Feature-Based Design and Manufacture' *Computer-Aided Design* Vol 25 No 1 (January 1993) pp 29-38

[Gib77] Giblin, P J, *Graphs, Surfaces and Homology* Chapman & Hall/Halsted Press, New York, NY, USA (1977)

[GZS88] Gossard, D C, Zuffante, R P and Sakurai, H, ' Representing Dimensions, Tolerances, and Features in MCAE Systems' *IEEE Computer Graphics & Applications* Vol 8 No 2 (1988) pp 51-59

[Hof93] Hoffmann, C M 'On the Semantics of Generative Geometry Representations' *Proceedings of the 19th ASME Design Automation Conf.* New York, NY, USA (1993) pp 411-419

[HS79] Haralick, R M and Shapiro, L G, 'The Consistent Labeling Problem: Part 1' *IEEE Trans. Pattern Matching & Mach. Intell.* Vol PAMI-1 No 2 (1979) pp 173-184

[Kri94] Kripac, J, 'Topological ID System' *Ph.D. Thesis* Czech Technical University in Prague, School of Electrical Engineering, Dept. of Computer Science, Czech Republic (1994)

[Mal90] Malmgvist, J 'A, Design System for Parametric Design of Complex Products' *Proc. 1990 ASME Advances in Design Automation Conf.* - DE-Vol 32-1 (1990) pp 17-24

[Mey88] Meyer, B, *Object-Oriented Software Construction* Prentice Hall International, UK (1988)

[PFP89] Pinilla, J M, Finger, S and Prinz, F B, 'Shape Feature Description and Recognition Using an Augmented Topology Graph Grammar' *Proc. NSF Engineering Design Research Conf.* College of Engineering, University of Massachusetts at Amherst, USA (11-14 June 1989) pp 285-300

[Rol91] Roller, D, 'An Approach to Computer-Aided Parametric Design' *Computer-Aided Design* Vol 23 No 5 (June 1991) pp 385-391

[Ros90] Rossignac, J R, 'Issues of Feature-Based Editing and Interrogation of Solid Models' *Computers & Graphics* Vol 14 No 2 (1990) pp 149-172

[RSV89] Roller, D, Shonek, F and Verroust, A, 'Dimension-Driven Geometry in CAD: A Survey' *Theory and Practice of Geometric Modeling* Springer-Verlag (1989) pp 509-523

[SL93] Sheu L C and Lin J T, 'Representation Scheme for Defining and Operating Form Features' *Computer-Aided Design* Vol 25 No 6 (June 1993) pp 333-347

[WO91] Wang, N and Ozsoy, M, 'A Scheme to Represent Features, Dimensions and Tolerances in Geometric Modeling' *J. Manuf. Syst.* Vol 10 No 3 (1991) pp 233-240